Fixed memory or attention?

Four small models on English→Finnish translation and school mathematics

Toni Huhtiniemi

October 2026

Abstract

We compare four small models (1.3–4.4M parameters) on two very different tasks, with the same width (256), the same depth (4 layers), the same optimizer and the same budget (one epoch, fp32). Three are classic RNN encoder-decoders (GRU, LSTM, minGRU): the input is squeezed into the final RNN states, and the decoder never sees the input again. The fourth is a Kimi Linear block that can look back at the input. On OPUS-100 English→Finnish, Kimi reaches **1.18 bits/byte** against **1.58** for the best RNN. On the DeepMind Mathematics dataset it answers **26.1%** of the questions exactly right against **17.6%**, and on the harder "extrapolation" questions **22.9%** against **9.3%**. The gain is largest when the answer can be built digit by digit from the question (sorting, rounding, adding), and near zero when it needs several hidden steps (equations, primes). With our simple code, Kimi is about 5× slower to train.

1 Introduction

Early neural translation models (Cho et al., 2014; Sutskever et al., 2014) read the input into one fixed-size memory and write the output from this memory alone; modern models *look back* at the input while they write. We ask how much the fixed memory costs on two tasks that need different things. **Translation** needs the *meaning* of the sentence, and small errors are allowed. **School mathematics** needs *exact symbols*: one wrong digit makes the whole answer wrong, and many answers must repeat long numbers from the question. We keep everything else the same: model size, depth, data handling, optimizer, schedule and seed.

2 Setup

Models. All models read and write raw UTF-8 bytes (256 values + 3 special tokens) and have width 256.

- **GRU** and **LSTM**: 4 encoder layers and 4 decoder layers. The final state of encoder layer l is the starting state of decoder layer l. After that the decoder only sees its own previous byte, so all information about the input must fit in this state: 1,024 numbers for GRU, 2,048 for LSTM (h and c).
- **minGRU** (Feng et al., 2024): the same design with 4 minGRU layers per side (1,024 numbers passed). Its gates depend only on the current input, so the recurrence is linear and trains with a fast parallel scan. With no hidden-to-hidden matrix it has only 1.25M parameters.
- **Kimi** (Kimi Team, 2025): a decoder-only Kimi Linear block with 4 layers *in total* (half as many as the RNNs): 3 Kimi Delta Attention (KDA) layers and 1 multi-head latent attention (MLA) layer, each followed by a SwiGLU MLP. KDA is a linear RNN with a matrix memory (4 heads × 64×64) updated by the gated delta rule. MLA is ordinary softmax attention over compressed keys and values, without positional encoding. The input is [question bytes, separator, answer bytes] and the loss is on the answer only. MLA can look at every input byte, so there is *no bottleneck*. Simplified: dense MLP instead of mixture-of-experts, plain PyTorch.

Data. *OPUS-100 En–Fi* (Zhang et al., 2020): pairs with at most 128 bytes on each side, 918k pairs. *Mathematics* (Saxton et al., 2019): 56 question types ("modules") in 8 areas: algebra, arithmetic, calculus, comparison, measurement, numbers, polynomials and probability. We take 12,000 random pairs from each module and difficulty (easy, medium, hard), 2.0M pairs in total, which is about 2% of the full training set. For both tasks the same 2,000 random pairs are held out for validation.

Math test. We use the two official test sets: *interpolate* (the same kind of questions as in training, 56 modules) and *extrapolate* (bigger numbers, longer lists, more steps; 15 modules). For every model we use the same 1,000 random questions per module and greedy decoding. An answer counts only if it matches the correct answer exactly. The score is the mean over modules.

Training. One epoch (OPUS: 3,579 steps, Math: 7,868 steps), batch 256, learning rate $2{\cdot}10^{-3}$ with 500 warm-up steps and cosine decay to 10%. Muon for hidden weight matrices, AdamW for embeddings, output layer and biases; weight decay 0.01, gradient clipping 1.0, fp32, seed 0. One NVIDIA RTX 2080 Super (8 GB). Translations use beam search (width 5).

3 Main results

1. **Attention wins on both tasks.** With about as many parameters as the GRU and half as many layers, Kimi's loss is 25% lower than the best RNN on OPUS and 21% lower on Math. On both tasks it reaches the RNNs' *final* loss in less than half of the training steps.

			OPUS En→Fi		Mathematics			
Model	Params	State passed	Val bits/byte	Time	Val bits/byte	Interpolate	Extrapolate	Time
GRU	3.36M	1,024	1.58	5 min	1.12	17.0%	8.1%	9 min
LSTM	4.41M	2,048	1.60	6 min	1.10	17.6%	9.3%	11 min
minGRU	1.25M	1,024	1.75	5 min	1.23	13.4%	5.8%	8 min
Kimi	3.43M	none (attends)	**1.18**	26 min	**0.87**	**26.1%**	**22.9%**	45 min

Table 1: Main results. Lower bits/byte is better, higher accuracy is better. Time is training time without evaluation.

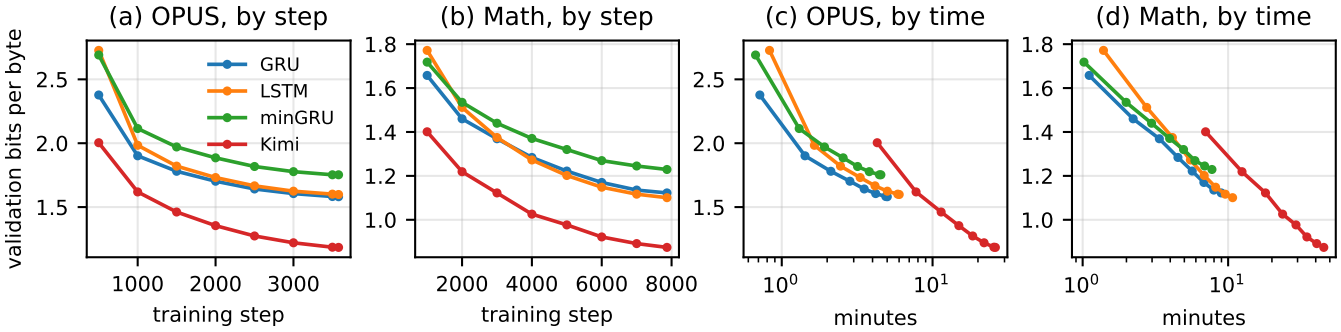


Figure 1: Validation loss during training. (a, b) By training step: Kimi is best at every point, on both tasks. (c, d) By wall-clock time (log scale): with our code, Kimi needs more time for the same loss early in training, and only its longer run gets it below the RNNs.

2. **The loss gain is similar, but the accuracy gain on math is much larger.** A 21% lower loss turns into 1.5× the interpolation accuracy and 2.5× the extrapolation accuracy. Exact match is strict: an answer with one wrong digit gets no credit, so a model that gets most digits right gains a lot.
3. **Among RNNs, GRU ≈ LSTM > minGRU.** GRU and LSTM are almost equal (GRU slightly better on OPUS, LSTM slightly better on Math), although the LSTM passes twice as many numbers. minGRU is last on both tasks, but it also has only a third of the parameters.
4. **But Kimi is slower in practice.** Training took about 5× longer (26 vs. 5 min on OPUS, 45 vs. 9 min on Math). At the GRU's full training time, Kimi was only at 1.93 bits/byte on OPUS (GRU: 1.58) and 1.33 on Math (GRU: 1.12). Part of this is our plain PyTorch code (the RNNs use cuDNN), but part is real: Kimi runs all its weights over both question and answer, an encoder-decoder runs each half of its weights over one side only.

4 Mathematics in detail

1. **The fixed memory cannot hold long numbers.** The RNNs often write an answer of the right form and length, with the right first digits, but the rest of the digits are wrong (Table 3). 1,024 numbers are not enough to store several long numbers exactly. Kimi can simply look back at them, so it wins clearly on sorting (83% vs. 12%), rounding (93% vs. 57%) and place value (93% vs. 71%).
2. **Attention also helps with digit-by-digit work.** Adding two numbers (33% vs. 1%), multiplying (32% vs. 6%) and differentiating (56% vs. 4%; for ab^2 the answer is $2a\,b$) need real computation, but each output digit depends only on a few input digits. With attention, the model can find those digits when it needs them.
3. **No gain when the answer needs hidden steps.** Primes stay at about 50% (guessing) and divisibility at about 70% for all models. Solving equations, mixed arithmetic, expanding polynomials and probability stay at 0–4%. The answers are short, so memory is not the problem: these need several hidden computation steps, which 4 small layers and one epoch cannot learn. On linear equations, all four models answer -10, -11 or -12 in about 70–80% of cases, whatever the question.
4. **Kimi handles harder questions better.** From interpolate to extrapolate, the RNNs lose about half of their accuracy, Kimi only 12%. Longer lists to sort: 55% vs. at most 1.4%. Adding numbers bigger than any seen in training: 18% vs. 0%.

5 Translation in detail

Table 4 shows the same pattern as math. The RNNs keep the *start* of the sentence ("Meidän pitäisi", "Voitko auttaa") and lose or replace the content words, sometimes with fluent but wrong Finnish ("my father has died"). Common openings seem easy to rebuild from a summary, while names of things (car, station, garden, environment) must be stored exactly, just like digits. Kimi keeps more of the content words; its errors are dropped words at the end, wrong word forms ("uusi" instead of "uuden") and repetition.

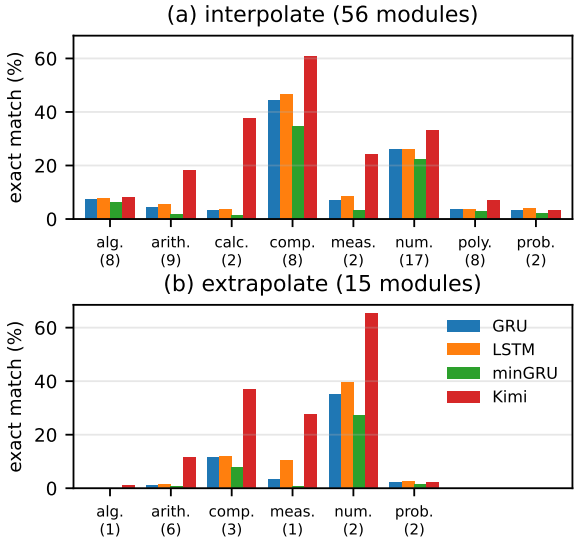


Figure 2: Math accuracy by area (number of modules in brackets). Colours as in Figure 1.

Test	Module	GRU	LSTM	minGRU	Kimi
interp.	comparison: sort	6.2	11.7	2.5	**83.1**
interp.	numbers: round	51.5	57.2	41.0	**93.4**
interp.	numbers: place value	71.2	68.1	44.7	**93.4**
interp.	calculus: differentiate	4.0	3.8	1.0	**55.7**
interp.	arithmetic: add/sub	0.5	0.8	0.1	**33.4**
interp.	arithmetic: multiply	4.1	6.2	1.8	**32.4**
interp.	numbers: is prime	52.9	51.9	47.8	53.0
interp.	numbers: is factor	70.1	72.0	68.6	72.7
interp.	algebra: linear 1d	2.9	3.7	2.5	2.7
interp.	arithmetic: mixed	0.6	0.8	0.1	1.0
interp.	polynomials: expand	0.4	0.3	0.1	0.3
extrap.	comparison: sort more	0.1	1.4	0.0	**55.1**
extrap.	numbers: round big	37.7	40.0	26.0	**85.3**
extrap.	arithmetic: add/sub big	0.0	0.0	0.0	**17.8**
extrap.	arithmetic: mixed longer	0.0	0.0	0.0	0.0

Table 2: Accuracy (%) on selected modules: the largest gains for Kimi (top), modules where no model learns anything (middle), and extrapolation (bottom).

Question	Correct	GRU	LSTM	minGRU	Kimi
Differentiate 13693705*b**2 + 2777934 with respect to b.	27387410*b	16978150*b	33793550*b	19175550*b	**27387410*b**
What is -109977604546359 minus -24? *(extrap.)*	-109977604546335	-10997760456445	-109797604545331	-1099756548791	-109977604546333
Sort 9, -365, 1, 4, 7, 3, 2, 5, -2 in descending order. *(extrap.)*	9, 7, 5, 4, 3, 2, 1, -2, -365	9, 5, 4, 3, 2, 1, -2, -3, -657	7, 4, 3, 2, 1, 2, 5, -3, -758	9, 3, 5, 2, 1, -2, -365, -77	**9, 7, 5, 4, 3, 2, 1, -2, -365**
Put 5, 55, 2, -5, -7, 3, 1, 469, 8 in descending order. *(extrap.)*	469, 55, 8, 5, 3, 2, 1, -5, -7	589, 65, 5, 4, 3, 1, -2, -7	645, 19, 5, 4, 5, 2, -3, -6, 7	685, 5, 3, 1, 2, -3, -5, -79	55, 469, 8, 5, 3, 2, 1, -5, -7
Solve 7*d + 24638 = 24743 for d.	15	-11	-11	-11	-11

Table 3: Answers from the test sets (correct answers in bold). The RNNs know the *form* of the answer but mix up, drop or invent digits. Kimi copies the digits; its errors are a wrong last digit and the order of 55 and 469. For the equation, all four models give the same guess.

English	GRU	LSTM	minGRU	Kimi
My sister bought a new car last week.	Sisareni kuuluu kaupungissa. *(“my sister belongs in the city”)*	Isäni on kuollut. *(“my father has died”)*	Minun pitäisi olla vain kaunis. *(“I should just be beautiful”)*	Sisareni osti uusi viime viikolla. *(“my sister bought new last week”; drops “car”)*
Can you help me find the train station?	Voitko kertoa minulle jotain muuta? *(“can you tell me something else?”)*	Voisitko auttaa minua? *(“could you help me?”)*	Voitko auttaa minua? *(“can you help me?”)*	Voitko auttaa minua? *(“can you help me?”)*
The children are playing in the garden.	Lapset ovat valmistautunut. *(“the children are prepared”, bad grammar)*	Poliisilla on täydellinen. *(“the police have a perfect”)*	Se on vaikeaa tapauksessa. *(broken)*	Lapset ovat leikkimässä. *(“the children are playing”; drops “garden”)*
We should protect the environment for future generations.	Meidän pitäisi käyttää selvää, että meidän pitäisi olla mahdollista. *(word salad)*	Meidän pitäisi pitää seuraavasti komissiolle. *(“we should keep as follows to the Commission”)*	Meidän pitäisi tehdä seuraavaksi. *(“we should do next”)*	Meidän pitäisi suojella tulevaisuuden tulevaisuudesta. *(“we should protect the future from the future”)*

Table 4: Translations of new everyday sentences (not in the training data). After one epoch all models are weak, but only Kimi keeps the main meaning.

On the time question, one more data point: in an earlier experiment, 1-layer RNNs of width 512 trained for 5 epochs (11–14 min, fp16) reached 1.44–1.53 bits/byte on the same OPUS split, far from Kimi’s 1.18 after one epoch. So more training time alone does not close the gap.

6 Limitations

One seed per model and one epoch only; all loss curves are still falling, so longer training would help every model. The models and the math training set are small, so the accuracies are far below published results (Saxton et al. used all 112M pairs). OPUS validation is a random split of the training file, and the translation examples are few and judged by hand by one non-native speaker. Kimi is a simplified re-implementation with slow kernels, and differs from the RNNs in more than attention (decoder-only, MLPs, normalisation). A cleaner test of attention alone would add attention (Bahdanau et al., 2014) to the GRU and change nothing else.

References. Bahdanau, Cho, Bengio (2014). Neural machine translation by jointly learning to align and translate. Cho et al. (2014). Learning phrase representations using RNN encoder-decoder. Sutskever, Vinyals, Le (2014). Sequence to sequence learning with neural networks. Feng et al. (2024). Were RNNs all we needed? Kimi Team (2025). Kimi Linear: an expressive, efficient attention architecture. Saxton, Grefenstette, Hill, Kohli (2019). Analysing mathematical reasoning abilities of neural models. Zhang et al. (2020). Improving massively multilingual NMT and zero-shot translation (OPUS-100). Jordan et al. (2024). Muon optimizer.